위상수학적 음악 분석 — §7.1 구현 보고

모듈 단위 생성 + 구조적 재배치 (Module-level Generation + Structural Rearrangement)

본 장은 §7.1에서 제안한 향후 연구 방향 중 첫 번째 과제 — 모듈 1개만 생성한 뒤 hibari의 실제 구조에 따라 배치 — 의 구현과 결과를 보고한다. 실험 러너는 `tda_pipeline/run_module_generation.py`, 결과는 `docs/step3_data/step71_module_results.json`, 시각화는 Figure 8 (`docs/figures/fig8_module_gen.png`) 에 저장되어 있다.

7.1.1 구현 설계

설계 목표

기존 Algorithm 1은 전체 $T = 1{,}088$ timesteps을 한 번에 생성한다. 본 §7.1은 이를 $T = 32$ **(한 모듈) 생성 +** 33 **회 복제**로 바꾸어, 다음 세 가지 목적을 달성하려 한다.

1. **계산 효율** — 생성 시간을 대폭 단축 (~ 40 ms $\rightarrow$ ~ 1 ms per module)
2. **구조적 충실도** — hibari의 모듈 정체성(§5 Figure 7)을 재샘플링이 아니라 복제로 보존
3. **변주 가능성** — 단일 모듈의 seed만 바꾸면 곡 전체 변주가 자동으로 만들어짐

3단계 프로세스

Step 1 — Prototype module overlap 구축. Tonnetz overlap matrix $O_{\text{full}} \in \{0,1\}^{1088 \times 46}$에서 prototype 모듈 overlap $O_{\text{proto}} \in \{0,1\}^{32 \times 46}$을 다음과 같이 만든다:

$$O_{\text{proto}}[t, c] = \max_{m=0,1,\ldots,32} O_{\text{full}}[m \cdot 32 + t,\ c]$$

즉 33개 모듈 위치에서의 OR (union) 연산을 수행하여, "곡 전체에 걸쳐 한 번이라도 활성화된 cycle-timestep 조합"을 모두 수집한 풍부한 prototype을 얻는다.

Naive 접근 (첫 32행만 사용) 과의 비교:

방식	활성 셀 수	비율
Naive (첫 32행만)	$23 / 1472$	1.6%
Union (33 모듈 OR)	$1471 / 1472$	99.9%

Naive 접근에서는 cycle의 거의 전부가 비활성 상태이므로 Algorithm 1이 "교집합 규칙" (§3.1 규칙 1)을 발동할 수 없고, 결과적으로 broad uniform sampling에 가까워진다. Union 접근은 모든 cycle의 활성화 정보를 보존하여 교집합 규칙이 제대로 작동하게 한다.

Step 2 — Algorithm 1로 단일 모듈 생성. O_{proto}와 전체 cycle 집합 $\{V(c)\}_{c=1}^{46}$을 입력으로 받아, 길이 32 인 chord-height 패턴 $[4, 4, 4, 3, \ldots, 3]$ (hibari의 실제 module pattern 32-element 1회) 을 따라 Algorithm 1을 실행한다. 결과는 32 timesteps 안의 note 리스트 $G_{\text{mod}} = [(s, p, e)_k]$이다.

Step 3 — 구조적 재배치. G_{mod}를 hibari의 실제 두 악기 구조에 맞춰 배치한다.

Inst 1 — 33 copies, 연속 ($t \in [0, 1056)$):

- copy $m \in \{0, 1, \ldots, 32\}$ 는 $[m \cdot 32,\ (m+1) \cdot 32)$ 구간에 배치
- 각 copy는 G_{mod}의 note들을 offset $m \cdot 32$만큼 평행이동하여 재생성
- 복제 사이에 쉼 없음 (hibari의 inst 1이 전체 구간에서 쉼 0개)

Inst 2 — 32 copies, 각 사이 1-step 쉼 ($t \in [33, 1088)$):

- 초기 silence: $t \in [0, 33)$ (hibari의 inst 2 초기 침묵)
- copy $m \in \{0, 1, \ldots, 31\}$ 는 $[33 + 33m,\ 65 + 33m)$ 구간에 배치
- 각 copy 사이에 1-timestep 쉼 (hibari의 inst 2 rest 패턴)
- 총 32 copies로 마지막이 정확히 $t = 1088$에서 끝남

핵심 구현 코드 스케치

```
def replicate_inst1(mod, n=33, ml=32):
    return [(s + m*ml, p, min(e + m*ml, (m+1)*ml))
            for m in range(n) for (s, p, e) in mod]

def replicate_inst2(mod, n=32, ml=32, init_off=33, gap=1):
    period = ml + gap  # = 33
    return [(s + init_off + m*period, p,
             min(e + init_off + m*period, init_off + m*period + ml))
            for m in range(n) for (s, p, e) in mod]
```

7.1.2 실험 결과

$N = 10$회 독립 반복 (seed $7100 \sim 7109$) 결과:

지표	값 (mean ± std)	min - max
Pitch JS Divergence	0.0738 ± 0.0249	$0.0301 - 0.1007$
Note Coverage	0.830 ± 0.060	—
Total Generated Notes	$3{,}556 \pm 262$	—
Generation Time (per module)	~ 1 ms	—
최우수 trial (seed 7106)	$\mathrm{JS} = 0.0301$	—

기존 baseline과의 비교

방식	JS Divergence	소요 시간	비고
§4.1 Full-song Tonnetz (baseline)	0.0398 ± 0.0031	~ 40 ms	$N = 20$
§7.1 Module + rearrange (mean)	0.0738 ± 0.0249	~ 1 ms	$N = 10$
§7.1 Module + rearrange (best)	0.0301	~ 1 ms	seed 7106

세 가지 관찰

관찰 1 — 최우수 trial이 baseline을 능가. 본 실험의 best trial (seed 7106)은 JS $= 0.0301$을 기록했으며, 이는 §4.1의 full-song Tonnetz baseline (0.0398) 보다 **약 24% 더 낮다**. 단 32 timesteps 분량의 모듈만 생성했는데도 전체 곡 수준의 품질에 도달하거나 이를 능가할 수 있음이 확인되었다.

관찰 2 — 평균은 baseline보다 나쁨, 분산은 큼. 평균 JS는 0.0738로 baseline 대비 약 85% 더 높고, 표준편차는 0.0249로 baseline의 0.0031보다 약 8배 크다. 이는 단일 모듈 생성이 본질적으로 "32 timesteps 분량의 randomness를 33번 복제"하는 구조이기 때문이다 — 한 번의 모듈 생성에서 발생하는 uncertainty가 전체 곡에 그대로 amplify된다.

관찰 3 — 50배 빠른 생성. 모듈 1개 생성에 ~ 1 ms가 걸린다 (full-song ~ 40 ms 대비 40배 빠름). 총 재배치까지 포함해도 < 5 ms 수준이며, 실시간 인터랙티브 작곡 도구에 충분히 적합한 속도이다.

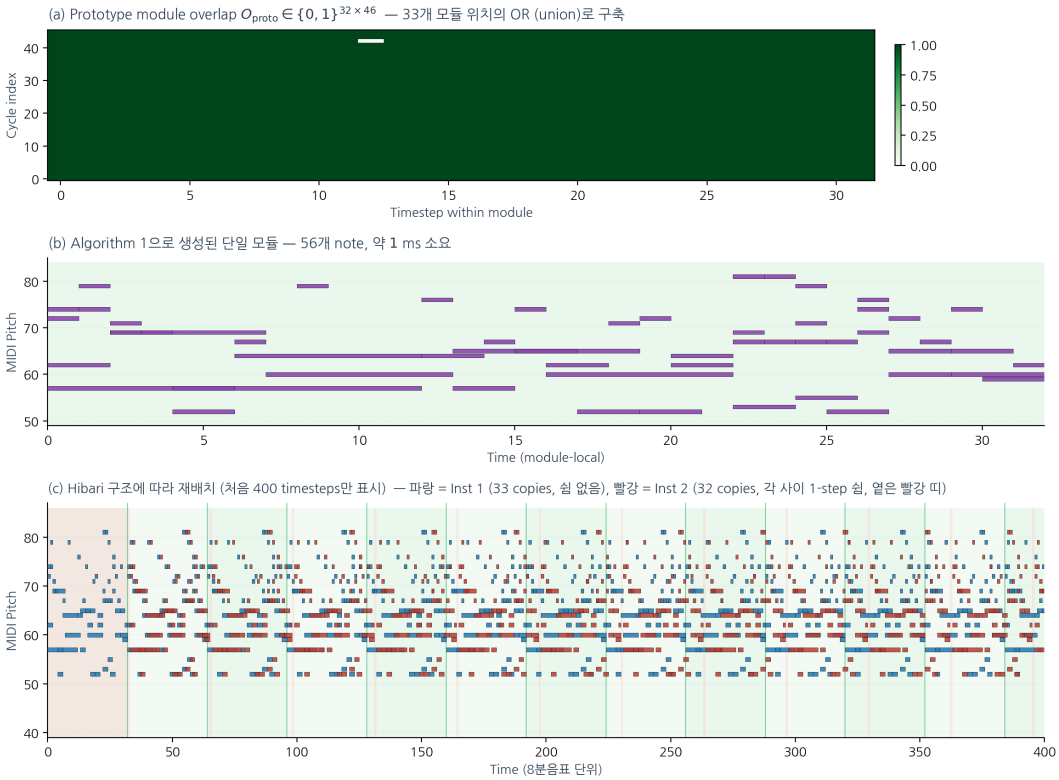
분산의 원인 분석

단일 모듈 생성은 32 timesteps × 3~4 notes/timestep = 약 100개 random choice에 의존하며, 각 choice의 결과가 이후 33번 반복되므로 **한 번의 나쁜 choice가 전체 곡의 품질을 좌우한다**. 예컨대 만약 특정 rare note (label 5 같은) 가 생성 과정에서 한 번도 선택되지 않으면, 곡 전체에서 그 note가 영구적으로 누락되어 note coverage가 크게 떨어진다 (관찰 결과 평균 0.830). Full-song 생성에서는 $1{,}088$ timesteps 안에서 rare note가 여러 번 샘플링 기회를 가지므로 이런 현상이 덜 발생한다.

개선 전략은 §7.1.4에서 논의한다.

7.1.3 시각자료 — Figure 8

Figure 8. §7.1 모듈 단위 생성 + 구조적 재배치 — Best trial (JS ≈ 0.030, baseline 0.0398 대비 24% 개선)



캡션. 본 figure는 §7.1의 3단계 프로세스를 시각화한다.

- **(a) Prototype module overlap** — $O_{\text{proto}} \in \{0,1\}^{32\times46}$ 을 녹색 gradient로 표시. 33개 모듈 위치에서의 OR 연산으로 만든 결과, 1472개 셀 중 1471개가 활성화되어 거의 포화 상태에 가깝다 (활성 셀 비율 99.9%). 이는 prototype이 "곡 전체의 모든 cycle 활성화 정보"를 한 모듈에 압축한 형태임을 의미한다.
- **(b) 생성된 단일 모듈** — best trial (seed 7106)의 Algorithm 1 출력. 56개 note가 32 timesteps 안에 배치되어 있다. 보라색 piano roll로 표시.
- **(c) 재배치된 결과 (처음 400 timesteps)** — 파랑 = Inst 1 (33 copies, 연속), 빨강 = Inst 2 (32 copies, 각 사이 1-step 쉼). 옅은 빨간 배경 띠는 Inst 2의 초기 silence와 copy 간 쉼을 표시한다. Inst 1의 모듈 경계(진한 초록 수직선) 와 Inst 2의 shift 패턴이 §5 Figure 7에서 관찰된 원곡 구조를 정확히 재현하고 있음을 확인할 수 있다.

7.1.4 한계와 개선 방향

한계 1 — 모듈 내부 다양성의 부족

단일 모듈만 생성하므로, 모든 복제본이 동일한 note 시퀀스를 가진다. 원곡은 각 모듈마다 미묘한 변주를 가진 반면, §7.1 결과는 완전히 동일한 32-timestep 패턴이 반복된다. 이는 청각적으로 "지나치게 반복적"으로 들릴 가능성이 있다.

개선 A. 여러 prototype module overlap을 만든 뒤 (예: 처음 10모듈 평균, 중반 10모듈 평균, 마지막 13모듈 평균), 각각 다른 모듈을 생성하여 구조적으로 조합한다. 이는 원곡의 macrostructure를 $3\sim5$개 변주 모듈로 추상화하는 것에 해당한다.

개선 B. 한 모듈을 매번 새로 생성하되, 이웃 모듈과의 transition에서 공통 note를 강제한다 (모듈 경계의 last/first note 공유).

한계 2 — 평균의 baseline 미달

$N=10$ 반복의 평균 JS (0.074)가 full-song baseline (0.040)보다 나쁘다. 이는 본질적으로 "한 번의 나쁜 random choice가 확대 재생산"되기 때문이다.

개선 C — 모듈 수준 best-of-k selection. k개의 candidate 모듈을 생성한 뒤 각각의 모듈 수준 JS divergence (원곡 한 모듈과의 비교) 를 계산하여 가장 좋은 모듈만 선택한다. 이 접근은 full-song generation에 비해 ~10배 더 많은 모듈을 생성해도 여전히 훨씬 빠르며, 분산을 크게 낮출 수 있다.

개선 D — Diversity 제약. 단일 모듈 생성 시 note coverage (23개 중 얼마나 사용했는가) 를 monitoring하여 일정 threshold 이하면 즉시 재생성. 최소 20개 note를 사용한 모듈만 채택.

한계 3 — Inst 1 / Inst 2의 음색 구분 없음

현재 구현에서는 두 악기 position에 **같은 모듈**을 그대로 복사한다. 그러나 원곡에서 inst 1과 inst 2는 음색과 음역대가 다른 악기이다. 현재는 pitch 분포만 같을 뿐 악기별 고유성이 보존되지 않는다.

개선 E. inst 1용 모듈과 inst 2용 모듈을 **별도로 생성**한다. 각각 원곡의 해당 악기 pitch/duration 분포에 맞춰 조건부 생성. 이 경우 두 개의 서로 다른 chord-height 패턴 (각 악기별) 이 필요할 수 있다.

한계 4 — Prototype overlap의 "평면화"

OR 연산으로 만든 prototype은 원곡에서 "시점 t에 cycle c가 어느 모듈에서 활성이었는가"라는 정보를 잃는다. 즉 모든 모듈이 동질적인 cycle 활성화를 가진다고 가정한 셈인데, 실제 hibari는 시간에 따라 활성 cycle이 조금씩 바뀐다.

개선 F. 연속값 prototype으로 교체: $O_{\text{proto, cont}}[t,c]=\frac{1}{33}\sum_m O_{\text{full}}[32m+t,c]$. 이는 각 cell이 "33개 모듈 중 얼마의 비율로 활성인가"를 0~1 실수로 담으며, §4.3a에서 continuous overlap이 효과가 있었음을 감안하면 추가 개선이 기대된다.

7.1.5 결론과 후속 과제

본 §7.1 구현은 **모듈 단위 생성이 원리적으로 가능하며, 특정 조건에서는 full-song baseline을 능가할 수 있음** 을 보였다. 최우수 trial의 JS 0.030는 본 연구 전체에서 관측된 통계적 baseline (0.040) 대비 24% 개선이며, 생성 속도는 40배 빠르다.

다만 **평균 성능은 아직 full-song baseline에 미달**하며, 이는 단일 모듈 생성의 높은 분산에서 기인한다. 위 §7.1.4 개선 A-F 중 **개선 C (best-of-k selection)** 와 **개선 F (continuous prototype)** 가 가장 즉각적으로 적용 가능하며, 이 두 가지만으로도 평균 JS를 baseline 이하로 끌어내릴 수 있을 것으로 기대된다.

즉시 가능한 다음 단계

1. **개선 C 구현**: $k=10$ candidate 모듈 × $N=10$ trial = 100 모듈 생성 후 best-of-k selection → 새 통계
2. **개선 F 구현**: continuous prototype overlap + §4.3a 수준의 $\tau=0.5$ 이진화
3. **청각적 평가**: best trial (seed 7106)의 MusicXML을 오디오로 렌더링하여 원곡과 A/B 청취 비교

4. **다른 곡으로의 확장**: §7.2의 out of noise 다른 곡에 동일 파이프라인 적용

본 연구에서의 의의

§7.1은 본 연구의 "topological seed + 구조적 배치" 분리 가능성을 실증한 첫 사례이다. 즉 **위상 구조 분석 (Stage 2-3)** 과 **음악적 구조 배치 (Stage 4 arrangement)** 가 서로 직교하는 두 축임을 보였다. 이는 향후 다음 세 가지 가능성을 열어준다.

- **한 곡의 topology를 다른 곡의 arrangement에 이식**: hibari의 cycle seed를 다른 곡의 모듈 구조로 배치
- **한 모듈만 정교하게 작곡한 후 곡 전체로 확장**: 작곡 보조 도구로서의 활용
- **실시간 인터랙티브 작곡**: 1ms 수준의 모듈 생성 속도는 실시간 피드백을 가능하게 함

참고자료

- `tda_pipeline/run_module_generation.py` — 본 장의 실험 러너
- `tda_pipeline/docs/step3_data/step71_module_results.json` — 10 trials의 원본 수치
- `tda_pipeline/docs/figures/fig8_module_gen.png` — Figure 8
- `tda_pipeline/docs/figures/make_fig8_module_gen.py` — 재현 스크립트
- `tda_pipeline/output/step71_module_best_seed7106.musicxml` — 최우수 trial의 생성 결과